

Template rapport technique – Assistant intelligent de recommandation d'événements culturels

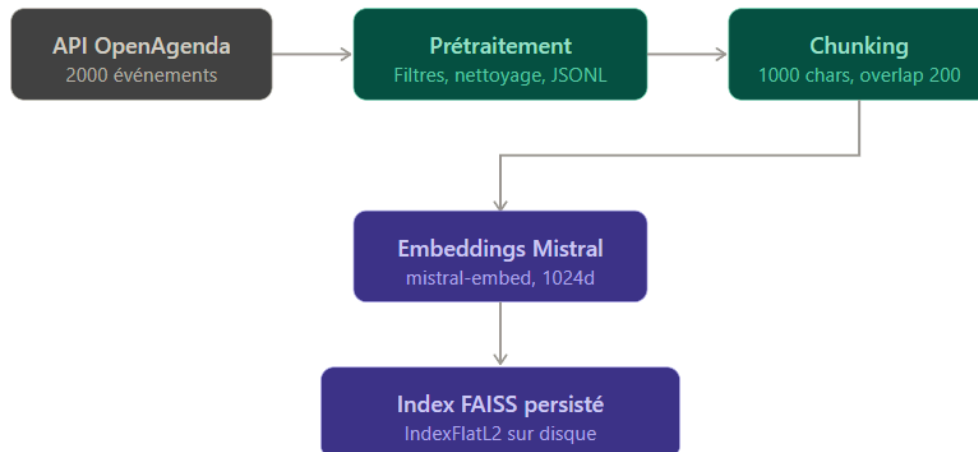
1. Objectifs du projet

- **Contexte** : mission pour Puls-Events, plateforme de recommandations culturelles. Besoin d'un chatbot qui répond aux questions des utilisateurs sur les événements à venir.
- **Problématique** : un LLM seul invente des événements et ne connaît pas le catalogue réel. Le RAG branche le LLM sur les vraies données, ce qui donne des réponses fiables et à jour.
- **Objectif du POC** : prouver la faisabilité technique (LangChain + Mistral + FAISS), la valeur métier (réponses utiles), la performance (mesurée avec Ragas)
- **Périmètre** :
 - Zone : Nouvelle-Aquitaine
 - Période : événements récents, 1 an d'historique (filtre sur la date de début)
 - Données : événements publics OpenAgenda (via le portail Opendatasoft)
 - Les événements annulés ou complets sont gardés, c'est au LLM de mentionner le statut.

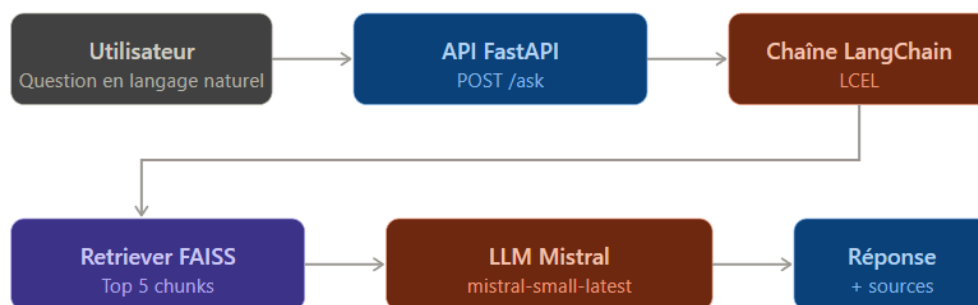
2. Architecture du système

- **Schéma global** (schéma UML) :

Phase offline : ingestion et indexation



Phase online : service API



Index restructurable à la demande via POST /rebuild (protégé)

- Étapes :
 1. Données : API OpenAgenda (Opendatasoft), événements bruts
 2. Prétraitement : nettoyage et structuration, puis documents
 3. Vectorisation : embeddings Mistral, puis index FAISS
 4. RAG : recherche FAISS + génération Mistral, orchestrés par LangChain
 5. Exposition : API FastAPI (/ask, /rebuild, /health)
- Technologies utilisées :
 - Récupération des données : requests + API Opendatasoft
 - Nettoyage : pandas / Python

- Embeddings : mistral-embed (1024 dimensions)
- Base vectorielle : faiss-cpu (recherche par similarité)
- Orchestration : LangChain (relie recherche et LLM)
- Génération : mistral-small-latest
- API : FastAPI + Uvicorn
- Évaluation : Ragas
- Conteneurisation : Docker + Compose

3. Préparation et vectorisation des données

- **Source** : dataset evenements-publics-openagenda via l'API Explore d'Opendatasoft.
 - Filtres : région = Nouvelle-Aquitaine et date de début sur les 365 derniers jours
 - Tri du plus récent au plus ancien, pagination par 100, maximum 10000 événements
- **Nettoyage** (preprocessing.py) :
 - suppression des balises HTML et des espaces multiples dans les descriptions
 - parsing des champs JSON pour garder le label français lisible (statut, modalité)
 - sélection des champs utiles uniquement
 - validation : on garde l'événement seulement s'il a un titre, une description et une date
 - déduplication par titre + ville
- **Chiffres** :
 - Événements bruts récupérés : 10000
 - Après nettoyage (filtres + validation + déduplication) : 7063
 - Distribution par état : Programmé 7022, Complet 24, Annulé 14, Reprogrammé 3
 - Top départements : Gironde 2906, Charente-Maritime 782, Pyrénées-Atlantiques 514, Vienne 473, Haute-Vienne 417
- **Chunking** (chunking.py) :

- outil : RecursiveCharacterTextSplitter
 - taille : 1000 caractères, chevauchement 200
 - raison : éviter des textes trop longs pour l'embedding et garder un contexte cohérent. La plupart des événements tiennent en un seul chunk
- **Embedding :**
 - modèle : mistral-embed (1024 dimensions)
 - les chunks sont envoyés par lots (géré automatiquement par le client Mistral)

4. Choix du modèle NLP

- **Modèle de génération :** mistral-small-latest (température 0.3).
- **Pourquoi ce modèle :**
 - imposé par la mission (Mistral) et intégration native avec LangChain
 - bon rapport qualité / coût
 - température basse (0.3) pour des réponses factuelles et peu d'invention
- **Prompting (prompt.py) :** prompt système qui cadre l'assistant
 - répondre uniquement à partir des événements fournis
 - ne pas inventer ; dire clairement si aucun événement ne correspond
 - répondre en français avec le titre, la date et le lieu
- **Limites du modèle :**
 - dépend de la qualité de la recherche (si le bon événement n'est pas retrouvé, il ne peut pas répondre)
 - peut mal interpréter certaines requêtes subjectives

5. Construction de la base vectorielle

- **FAISS :** index créé avec FAISS.from_documents (index plat, similarité L2 par défaut).

- **Persistance :**
 - méthode : `save_local()` dans le dossier `data/index/`
 - format : deux fichiers, `index.faiss` (vecteurs) et `index.pkl` (textes et métadonnées)
 - rechargement : `load_local()`
- **Métadonnées conservées par document (`event_to_document.py`) :**
 - identité : uid, titre, url
 - dates : date de début, date de fin, plage de dates lisible
 - lieu : ville, département, région, code postal
 - pratique : statut, modalité, âges min/max, accessibilité, image, lien en ligne
- **Chiffres :**
 - Documents LangChain créés : 7063
 - Chunks indexés : 10 551 (moyenne 1.49 chunk par événement)
 - Modèle d'embeddings : mistral-embed (1024 dimensions)

6. API et endpoints exposés

- **Framework :** FastAPI + Uvicorn. Documentation interactive Swagger sur `/docs`.
- **Endpoints :**
 - **GET /health :** vérifier que l'API répond
 - **POST /ask :** question, puis réponse + sources
 - **POST /rebuild :** recharger les données et reconstruire l'index (protégé par une clé)

- **Format de la requête /ask :** `{"question": "..."}`
- **Format de la réponse /ask :** `{"answer": "...", "sources": [{uid, title, city, url}]}`

- **Exemple d'appel :**

```
curl -X POST http://localhost:8000/ask -H "Content-Type: application/json" -d '{"question": "Quels concerts de jazz a Bordeaux ?"}'
```

- **Performance :** l'index est chargé une seule fois au démarrage

- **Gestion des erreurs :**
 - question vide : 422
 - index non construit : 503
 - mauvaise clé sur /rebuild : 401
 - erreur de génération : 500
- **Tests :** tests automatiques de l'API dans tests/test_api.py (santé, question valide, question vide, protection de /rebuild).

7. Évaluation du système

- **Jeu de test annoté (data/eval/questions.jsonl) :**
 - 10 questions / réponses de référence
 - annotation manuelle, à partir des vraies données indexées (cas variés : jazz, enfants, ville hors zone, gratuit, science, etc.)
- **Métriques (Ragas, juge = mistral-small) :**
 - **Faithfulness** : la réponse est-elle bien appuyée sur le contexte ?
 - **Answer relevancy** : la réponse colle-t-elle à la question ?
 - **Context precision** : les documents retrouvés sont-ils pertinents ?
- **Résultats obtenus :**
 - **Faithfulness : 0.830**
 - **Answer relevancy : 0.830**
 - **Context precision : 0.820**
- **Lecture :** le système répond bien aux questions (relevancy élevée) et retrouve des documents plutôt pertinents (precision correcte). Les 3 métriques sont autour de 0.83. 9 questions sur 10 ont une faithfulness de 0.83 à 1.0, mais « concerts à Paris » est à 0 car Ragas pénalise les refus (il ne comprend pas une absence normale).
- **Analyse qualitative :**

Bonne réponse - question avec réponse ciblée

Question : "Quels concerts de jazz à Bordeaux ?"

Réponse : "Il y a Les Vendredis chez Calixte le 19 juin 2026, Petit concert de Jazz, Quatuor Trombones Jazz à Bordeaux"

Faithfulness 0.83, context precision 1.0

Le système identifie correctement les événements et les cite.

Réponse faible - question hors périmètre bien refusée, score bas

Question : "Y a-t-il des concerts à Paris ?"

Réponse : "Il n'y a aucun concert prévu à Paris parmi les événements disponibles. Les concerts proposés ont lieu en Nouvelle-Aquitaine"

Faithfulness : 0.0

Le système identifie correctement que la question sort du périmètre et refuse explicitement, sans inventer d'événements à Paris. Il n'hallucine pas.

8. Recommandations et perspectives

- **Ce qui fonctionne bien :**
 - réponses fidèles aux données, refus correct quand l'information n'existe pas (ex: concerts à Paris)
 - pipeline reproductible (scripts + Docker)
- **Limites du POC :**
 - volumétrie limitée (max 10000 événements, 1 région)
 - coût et latence dépendants de l'API Mistral
 - jeu d'évaluation calé sur des données à l'instant T (si on relance /rebuild, les réponses de référence peuvent dériver)
- **Améliorations possibles :**
 - élargir la zone et la volumétrie
 - ajouter l'historique de conversation
 - mise en production : index managé, cache, authentification renforcée, CI/CD

9. Organisation du dépôt GitHub

- Arborescence :

```
puls-events-rag/  
  src/  
    data/      récupération + nettoyage OpenAgenda  
    vectorstore/ documents, chunking, index FAISS  
    rag/       retriever, prompt, generator, chain  
    api/       application FastAPI  
  scripts/     fetch_data, build_index, evaluate_rag  
  tests/       tests unitaires (pytest)  
  data/eval/   jeu de test annoté  
  docs/        rapport + schéma + slides  
  Dockerfile, docker-compose.yml  
  pyproject.toml
```

- Rôle de chaque dossier :
 - src/data : télécharge les événements et les nettoie
 - src/vectorstore : transforme en documents, découpe, construit l'index FAISS
 - src/rag : la chaîne question, recherche, réponse
 - src/api : expose en HTTP
 - scripts : commandes pour reconstruire les données, l'index, et lancer l'évaluation
 - tests : valident chaque brique

10. Annexes (exemples)

- Extraits du jeu de test annoté
 - {"id": 1, "question": "Quels concerts de jazz à Bordeaux ?",
"reference_answer": "Il y a quelques concerts de jazz à Bordeaux, souvent autour de la fête de la musique : Les Vendredis chez Calixte (reprises jazz par des élèves), un Petit concert de Jazz de l'Orchestre d'Harmonie, le Quatuor Trombones Jazz à la Fête cuivrée, ou les examens de jazz du conservatoire. Plutôt de petits formats, souvent en entrée libre."}
- Prompt utilisé (si spécifique)
 - voir src/rag/[prompt.py](#)

- Extraits de logs ou exemples de réponse JSON

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/ask' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "question": "Quels concerts de jazz a Bordeaux ?"
  }'
```

Request URL

http://localhost:8000/ask

Server response

Code	Details
200	Response body <pre>{ "answer": "Voici les concerts de jazz à Bordeaux parmi les événements disponibles :\n\n- **Concert du dimanche**\n *Dimanche 3 janvier 2027, 18h00*\n Auditorium de Bordeaux (Gironde)\n *C lassic jazz avec des arrangements originaux et du swing.*\n\n- **Belmondo X Belmondo**\n *Mardi 8 décembre 2020, 20h00*\n Auditorium de Bordeaux (Gironde)\n *Jazz, pop et musique du monde a vec des thèmes de films.*\n\n- **Shai Maestro**\n *Samedi 28 novembre 2020, 20h00*\n Auditorium de Bordeaux (Gironde)\n *Jazz, pop et musique du monde, centré sur l'esprit du piano.*", "sources": [{ "uid": "89767981", "title": "Concert du dimanche", "city": "Bordeaux", "url": "https://openagenda.com/ville-de-bordeaux/events/concert-du-dimanche-9755601" }, { "uid": "5483072", "title": "Belmondo X Belmondo", "city": "Bordeaux", "url": "https://openagenda.com/ville-de-bordeaux/events/belmondo-x-belmondo" }, { "uid": "58923977", "title": "Mardis de la musique : Influences klezmer et créole dans la musique jazz et moderne", "city": "Bordeaux", "url": "https://openagenda.com/ville-de-bordeaux/events/mardis-de-la-musique-influences-klezmer-et-creole-dans-la-musique-jazz-et-moderne" }, { "uid": "80451397", "title": "Shai Maestro", "city": "Bordeaux", "url": "https://openagenda.com/ville-de-bordeaux/events/shai-maestro-8176341" }] }</pre> Download Response headers <pre>content-length: 1468 content-type: application/json date: Thu, 11 Jun 2026 18:06:32 GMT server: uvicorn</pre>